R guide for GENG5507 midsemester revision

Use this alongside the statistics cheat sheet, the question bank and worked examples. This guide covers the R commands in the supplied statistics slides, lecture question PDFs and Practice Sheets 1–5. Extra commands that make the examples complete are labelled. Code examples are illustrative unless a practice-sheet question is identified.

Run code in RStudio: type in the Console, or select script lines and press Ctrl+Enter (Cmd+Enter on macOS). Read a result by entering the object name. R is case-sensitive. Run each fenced block from its first line; later lines in that block may use its setup.

1. Decide what the question needs

Asked for	Use	Main trap
Probability of exactly k discrete events	`d...`	A continuous density height is not a probability.
Probability below a boundary	`p...`	Default is $P(X \leq x)$.
Probability above a boundary; reliability	`p...(..., lower.tail = FALSE)`	For discrete X, "at least k" needs boundary $k-1$.
Boundary with a specified probability	`q...`	Default probability is the lower-tail area.
Random observations or simulation	`r...`	First argument is the number of draws.
Sample mean and variability	`mean`, `var`, `sd`	`sd` uses $n-1$; standard error is $\frac{s}{\sqrt{n}}$.
Estimated survival with suspensions	`Surv`, `survfit`	A suspension is a censored observation, not a failure.

2. Syntax, vectors and small mathematical tools

```
x <- c(2, 4, 8)              # c(): combine values
x[1]                        # 2; indexing starts at 1
x[x > 3]                    # 4, 8; logical selection
x^2                         # elementwise powers: 4, 16, 64
x == 4                      # equality test; assignment uses <-
1:4                         # integer sequence
length(c(2, 4, 8))           # 3 observations
sum(c(2, 4, 8))              # 14
sqrt(9)                     # 3
exp(0)                      # 1; exp(x) = e^x
log(exp(2))                 # 2; log() means natural logarithm
gamma(3)                   # 2; Gamma(3) = 2!
```

Use * for multiplication, `/` for division and parentheses to make the numerator and denominator clear. `&`, `|` and `!` mean elementwise AND, OR and NOT. `NA` means missing, `Inf` means infinity. A comment starts with `#`.

Helpful extras, each with a minimal example:

```
abs(-2)                     # 2
choose(10, 2)               # 45 ways to choose two from ten
factorial(4)                # 24
ceiling(59.72)              # 60; round UP a required sample size
floor(2.8)                  # 2
round(0.123456, 3)          # 0.123; round only the final answer
seq(0, 3, by = 1.5)         # 0, 1.5, 3
rep(5, times = 3)           # 5, 5, 5
prod(c(0.9, 0.8))           # 0.72; product
sort(c(8, 2, 4))            # 2, 4, 8
min(c(2, 4, 8))             # 2
max(c(2, 4, 8))             # 8
range(c(2, 4, 8))           # 2, 8
is.na(c(2, NA))             # FALSE, TRUE
mean(c(2, NA, 4), na.rm = TRUE) # 3
```

Define a function; vectorise a piecewise formula

`function()` defines a reusable calculation. `if` handles one condition; the helpful extra `ifelse()` handles a vector of conditions.

```r
square <- function(x) { x^2 }
square(3)                  # 9
ifelse(c(-1, 2) > 0, 1, 0) # 0, 1
```

Practice Sheet 2 uses $f(x) = xe^{-x}$ for $x > 0$. Include its support when implementing it:

```r
f_X <- function(x) ifelse(x > 0, x * exp(-x), 0)
F_X <- function(x) ifelse(x > 0, 1 - (1 + x) * exp(-x), 0)
f_X(2)                     # density height, about 0.2707
F_X(3) - F_X(1)            # P(1 < X <= 3), about 0.5366
plot(f_X, from = 0, to = 8, ylab = "f(x)",
     main = expression(f(x) == x * e^{-x}))
```

`expression(f(x) == x^2)` is a minimal example of `expression()`: it creates mathematical plot annotation, rather than calculating a probability.

3. Summaries and displays

Each course summary function gets a separate minimal call:

```r
mean(c(2, 4, 6))           # 4
median(c(2, 4, 20))        # 4
var(c(2, 4, 6))            # 4; sample variance
sd(c(2, 4, 6))             # 2; sample standard deviation
summary(c(2, 4, 6))        # minimum, quartiles, median, mean, maximum
head(c(2, 4, 6), 2)        # first two entries
class(c(2, 4, 6))          # "numeric"
options(digits = 6)        # display precision; does not round stored data
```

For complete observations x_i, the normal MLE uses n in the variance denominator. Do not confuse that with the sample variance:

```r
x <- c(10, 17, 24, 30, 14, 18, 15, 13, 21)
mu_hat <- mean(x)
sigma_hat_mle <- sqrt(mean((x - mu_hat)^2))
s_sample <- sd(x)
c(mu_hat = mu_hat, sigma_mle = sigma_hat_mle, sample_sd = s_sample)
```

Plotting functions: minimal examples

```r
hist(c(2, 3, 3, 5, 8))       # histogram; counts by default
boxplot(c(2, 3, 3, 5, 8))    # median, quartiles, whiskers/outliers
stem(c(2, 3, 3, 5, 8))       # text stem-and-leaf display
plot(c(1, 2, 3), c(2, 4, 3)) # scatterplot
curve(x^2, from = 0, to = 2) # graph a formula
text(1, 2, "annotation")     # add a label to the current plot
```

`lines()` adds to an existing plot; `density()` estimates a smooth density from observations:

```r
x <- c(1, 2, 2, 3, 4, 5, 6)
density(x)                   # density() minimal example
hist(x, freq = FALSE)
lines(density(x), col = "red") # lines() minimal example
```

Practice Sheet 1 normal simulation/overlay pattern:

```r
set.seed(5507)               # repeat the same pseudorandom sequence
x <- rnorm(1000, mean = 20, sd = 5)
boxplot(x)
hist(x, freq = FALSE, main = "Normal simulation", xlab = "x")
curve(dnorm(x, mean = 20, sd = 5), from = 5, to = 35,
      add = TRUE, col = "red")
text(26, 0.08, "N(20, 5^2) PDF", col = "red")
```

`freq = FALSE` gives **density**, so histogram area is one. It does not make every bar height a relative frequency. `nclass`/a single-number `breaks` is a suggested bin count; for an exact bin width of 1.5, supply explicit boundaries (Practice Sheet 4 style):

```r
x <- c(0.8, 1.6, 2.7, 3.2, 4.9, 6.1)
hist(x, breaks = seq(0, 7.5, by = 1.5), freq = FALSE)
```

4. Distribution functions: every prefix with a minimal call

The course explicitly uses `dnorm`, `pnorm`, `qnorm`, `rnorm`, `pbinom`, `qbinom`, `rbinom`, `dexp`, `rexp`, `dweibull`, `rweibull`, `rlnorm`, `pt` and `qt`. The remaining family members below are helpful extensions for calculating the distributions in the statistics notes.

Family	Parameter interpretation
Normal	`mean = mu`, **`sd = sigma`, not variance**.
Binomial	`size = n` trials, `prob = p` success chance. Bernoulli uses `size = 1`.
Poisson	`lambda` is the expected count over the specified interval.
Exponential	`rate = lambda`; mean $\frac{1}{\lambda}$.
Weibull	`shape = beta`, `scale = eta`; eta is characteristic life, not usually mean.
Lognormal	`meanlog` and `sdlog` describe $\log X$, not X.
Gamma	`shape = a`, `rate = lambda`, or `scale = 1/lambda`; never specify both.
Uniform	`min = a`, `max = b`.
Beta	`shape1 = a`, `shape2 = b`; support $[0, 1]$.
Student's t	`df` is degrees of freedom, usually $n - 1$ for a sample mean.

Within every block: line 1 is a density/PMF height, line 2 a cumulative probability, line 3 a lower-tail quantile, line 4 five random draws.

```r
# Normal N(3, 2^2)
dnorm(3, mean = 3, sd = 2)
pnorm(3.5, mean = 3, sd = 2)
qnorm(0.9, mean = 3, sd = 2)
rnorm(5, mean = 3, sd = 2)

# Binomial(10, 0.08)
dbinom(2, size = 10, prob = 0.08)
pbinom(2, size = 10, prob = 0.08)
qbinom(0.95, size = 10, prob = 0.08)
rbinom(5, size = 10, prob = 0.08)

# Poisson: mean count 3
dpois(2, lambda = 3)
ppois(2, lambda = 3)
qpois(0.95, lambda = 3)
rpois(5, lambda = 3)

# Exponential: mean lifetime 5
dexp(5, rate = 0.2)
pexp(5, rate = 0.2)
qexp(0.9, rate = 0.2)
rexp(5, rate = 0.2)

# Weibull: beta = 2, eta = 5
dweibull(5, shape = 2, scale = 5)
pweibull(5, shape = 2, scale = 5)
qweibull(0.9, shape = 2, scale = 5)
rweibull(5, shape = 2, scale = 5)

# Lognormal: log(X) ~ N(2, 0.2^2)
dlnorm(8, meanlog = 2, sdlog = 0.2)
plnorm(8, meanlog = 2, sdlog = 0.2)
qlnorm(0.9, meanlog = 2, sdlog = 0.2)
rlnorm(5, meanlog = 2, sdlog = 0.2)

# Gamma: shape 2, rate 1
dgamma(2, shape = 2, rate = 1)
pgamma(2, shape = 2, rate = 1)
qgamma(0.9, shape = 2, rate = 1)
rgamma(5, shape = 2, rate = 1)
```

```r
# Continuous uniform on [0, 10]
dunif(3, min = 0, max = 10)
punif(3, min = 0, max = 10)
qunif(0.9, min = 0, max = 10)
runif(5, min = 0, max = 10)

# Beta(2, 3)
dbeta(0.5, shape1 = 2, shape2 = 3)
pbeta(0.5, shape1 = 2, shape2 = 3)
qbeta(0.9, shape1 = 2, shape2 = 3)
rbeta(5, shape1 = 2, shape2 = 3)

# Student's t: 13 degrees of freedom
dt(2.16, df = 13)
pt(2.16, df = 13)
qt(0.975, df = 13)
rt(5, df = 13)
```

Discrete uniform sampling uses `sample()` rather than `runif()`:

```r
sample(1:6, size = 5, replace = TRUE) # five independent fair-die rolls
```

Tail, interval and quantile patterns from the practice questions

```r
# Practice Sheet 1: normal probabilities and quantiles
pnorm(2, lower.tail = FALSE)          # P(Z > 2) = 0.0227501
pnorm(0.57) - pnorm(0.2)              # P(0.2 < Z < 0.57) = 0.136401
qnorm(0.975)                          # central 95% boundary = 1.959964
pnorm(1, 3, 2) + pnorm(5, 3, 2, lower.tail = FALSE) # P(|X-3| >= 2)
qnorm(0.1, 3, 2, lower.tail = FALSE) # P(X > x) = 0.1: x = 5.563103

# Practice Sheet 2: ten resistors, defect probability 0.08
pbinom(1, 10, 0.08, lower.tail = FALSE) # P(X >= 2)
pbinom(4, 10, 0.08) - pbinom(1, 10, 0.08) # P(2 <= X <= 4)

# Practice Sheet 5: t-table questions
pt(2.160, df = 13)                    # 0.974983
pt(1.943, df = 6, lower.tail = FALSE) # 0.0500125
pt(2.750, df = 30) - pt(-2.750, df = 30)
qt(0.01, df = 29)                     # -2.462021
```

For continuous distributions, including or excluding an endpoint gives the same probability. For discrete distributions it can change the answer. `qbinom(p, ...)` returns the smallest integer k for which $P(X \leq k) \geq p$; exact equality to the requested probability is not generally possible.

5. Reliability, hazard and lifetime estimates

Reliability $R(t) = P(T > t)$ is an upper tail. For a nonnegative lifetime, MTTF is the population mean $E[T]$. Hazard is $h(t) = \frac{f(t)}{R(t)}$, a rate rather than a probability.

```r
# Weibull example: beta = 2, eta = 5
beta <- 2
eta <- 5
time <- 5
reliability <- pweibull(time, beta, eta, lower.tail = FALSE)
hazard <- dweibull(time, beta, eta) / reliability
mttf <- eta * gamma(1 + 1 / beta)
sigma <- eta * sqrt(gamma(1 + 2 / beta) - gamma(1 + 1 / beta)^2)
B10 <- qweibull(0.1, beta, eta) # 10% failed, 90% surviving
c(R = reliability, h = hazard, MTTF = mttf, SD = sigma, B10 = B10)

# Exponential lifetime with rate 0.2
pexp(5, rate = 0.2, lower.tail = FALSE) # R(5)
1 / 0.2                                 # MTTF = 5

# Independent components at a specified time
R_components <- c(0.9, 0.8, 0.95)
```

```r
prod(R_components)                    # series reliability
1 - prod(1 - R_components)            # active parallel reliability
```

Complete-data model estimates (illustrative observations):

```r
x <- c(2, 3, 4, 6, 8, 10)
1 / mean(x)                           # exponential rate MLE
mean(log(x))                          # lognormal meanlog MLE
sqrt(mean((log(x) - mean(log(x)))^2))    # lognormal sdlog MLE
```

6. Import, inspect and export data

The course refers to `microwaves.txt`, `streetlamps.xlsx` and failure-data CSVs. Those external datasets are not included in this workspace. Do not interpret the small data below as those real files or as their answers.

Self-contained minimal examples

```r
getwd() # relative paths start here
read.csv(text = "time,status\n2,1\n5,0")
read.table(text = "time status\n2 1\n5 0", header = TRUE)

# data.frame() is a helpful extra: make a table without an external file
failure_data <- data.frame(time = c(2, 5, 8), status = c(1, 0, 1))
head(failure_data)
class(failure_data)                     # "data.frame"
class(failure_data$time)                # "numeric"
names(failure_data)                     # column names
summary(failure_data)
subset(failure_data, status == 1)       # failures only for inspection
failure_data$time                       # numeric column
failure_data[failure_data$status == 1, ] # equivalent row selection

# names() also renames a column, as in the streetlamps screenshot
streetlamps <- data.frame(id = 1:3, old_name = c(2, 5, 8))
names(streetlamps)[2] <- "Time"
streetlamps$Time

# tempfile() is a helpful extra: safe disposable export destinations
csv_path <- tempfile(fileext = ".csv")
txt_path <- tempfile(fileext = ".txt")
write.csv(failure_data, csv_path, row.names = FALSE)
write.table(failure_data, txt_path, sep = "\t", row.names = FALSE)
read.csv(csv_path)
read.table(txt_path, header = TRUE)
```

External-file examples: require the named file and optional package

These commands are intentionally commented so the rest of this document can run without missing LMS files:

```r
# microwaves <- read.table("microwaves.txt", header = TRUE)
# failure_data <- read.csv("exa1.csv")
# install.packages("readxl")        # install once, with internet access
# library(readxl)                   # load each new R session
# streetlamps <- readxl::read_xlsx("streetlamps.xlsx")
# head(streetlamps)
# names(streetlamps)[2] <- "Time"   # inspect names first
```

`read_xlsx("streetlamps.xlsx")` is the exact function used in the Practice Sheet 4 screenshot. `readxl::read_excel("streetlamps.xlsx")` is a helpful alternative supporting both XLS and XLSX. `package::function` selects a function without attaching the package. Minimal setup calls are `install.packages("survival")` (installation, only when needed) and `library(survival)` (load).

7. Suspensions, Kaplan–Meier and Weibull fitting

Status coding: `1` = observed failure, `0` = right-censored/suspended. A censored unit survived at least its recorded time. Use numeric 0/1, not factors or ambiguous text labels.

Exponential MLE with right-censoring

Practice Sheet 4's total-time calculation assumes an exponential model, not an arbitrary lifetime distribution:

$$\hat{\lambda} = \frac{d}{\sum_i t_i}, \qquad \widehat{\mathrm{MTTF}} = \frac{\sum_i t_i}{d}, \quad d = \sum_i \mathrm{status}_i.$$

```
failure_data <- data.frame(time = c(2, 4, 5, 7, 8, 10),
                           status = c(1, 1, 0, 1, 0, 1))
d <- sum(failure_data$status)
exposure <- sum(failure_data$time)
c(rate = d / exposure, MTTF = exposure / d) # 0.111111, 9
```

Dropping suspended units loses exposure; pretending they failed distorts the likelihood. If no failures are observed, there is no finite MTTF estimate from this formula.

`Surv()`, `survfit()`, `summary()` and `plot()`

Requires the `survival` package (available in the environment used to check these notes). This block creates all its data:

```
library(survival)
Surv(c(2, 5), c(1, 0))       # minimal Surv(): 5+ denotes censored at 5
failure_data <- data.frame(time = c(2, 4, 5, 7, 8, 10),
                           status = c(1, 1, 0, 1, 0, 1))
fit <- survfit(Surv(time, status) ~ 1, data = failure_data)
summary(fit, times = c(3, 6, 9)) # estimated R(t): 5/6, 2/3, 4/9
plot(fit, xlab = "Time", ylab = "Estimated reliability R(t)",
     conf.int = TRUE)
plot(Surv(failure_data$time, failure_data$status)) # slide shortcut also works
```

`~ 1` means one overall survival curve without groups. Kaplan–Meier does not assume exponential or Weibull lifetimes; interpreting censoring requires noninformative censoring. Query actual observed time ranges. In the real Practice Sheet 4 dataset the requested times are 20, 50 and 100; use `summary(fit, times = c(20, 50, 100))` after fitting that dataset. The six-row example above does not cover those times.

`weibull.mle()`: course command, uncensored observations only

Optional package example; not executed during verification:

```
# install.packages("weibullness")
# library(weibullness)
# weibullness::weibull.mle(c(2, 3, 4, 6, 8, 10), threshold = 0)
```

Setting `threshold = 0` fits a two-parameter Weibull. **Source correction:** the course's `weibull.mle(q3data$time, 0)` passes only times, so it treats every time as a failure. It does not account for suspensions. Fitting only the subset of failures also discards censoring information. The documented function has no `status` argument. CRAN: weibull.mle.

Helpful extension: correctly fit a Weibull with censored observations

`survreg()` fits an accelerated-failure-time parameterisation; convert its parameters before using `pweibull()`. Its `scale` is the reciprocal of Weibull shape, not the Weibull lifetime scale. The conversion below follows the package documentation. CRAN: survreg.

```
library(survival)
failure_data <- data.frame(time = c(2, 4, 5, 7, 8, 10),
                           status = c(1, 1, 0, 1, 0, 1))
fit_w <- survreg(Surv(time, status) ~ 1, data = failure_data,
                 dist = "weibull")             # survreg() minimal fit
coef(fit_w)                                    # coef(): intercept
beta_hat <- 1 / fit_w$scale
eta_hat <- exp(coef(fit_w)[1])
pweibull(5, shape = beta_hat, scale = eta_hat, lower.tail = FALSE)
eta_hat * gamma(1 + 1 / beta_hat)              # fitted MTTF
```

`estimate_cdf()` and `plot_prob()`: current package syntax

The course shows an older vector-based `estimate_cdf(time, status=...)` call. The current documented API expects a `reliability_data()` object. Its `"kaplan"` plotting positions use a modification of Kaplan–Meier; use `survival::survfit()` for the standard survival estimate above. CRAN: estimate_cdf.

Optional package block; each of the three functions has its own minimal call after setup. Not executed during verification:

```
# install.packages("weibulltools")
# library(weibulltools)
# wt_data <- weibulltools::reliability_data(
#   x = c(2, 4, 5, 7, 8, 10), status = c(1, 1, 0, 1, 0, 1))
# cdf_est <- weibulltools::estimate_cdf(wt_data, methods = "kaplan")
# weibulltools::plot_prob(cdf_est, distribution = "weibull")
```

`reliability_data()` constructs the required data class; `plot_prob()` draws estimated failure probabilities on transformed distribution axes. Approximately straight points support the chosen distribution as a candidate, but are not proof. CRAN: reliability_data, CRAN: plot_prob.

8. Sample means, standard errors, Z and t

Practice Sheet 5 Q2: carbide tips have population standard deviation 12; a sample of 36 has standard error 2. Find the probability the sample mean is within 4 of the true mean, then the sample size for 99% coverage:

```
sigma <- 12
n <- 36
error <- 4
SE <- sigma / sqrt(n)
2 * pnorm(error / SE) - 1             # 0.9544997
ceiling((qnorm(0.995) * sigma / error)^2) # 60 tips
```

For iid normal observations and unknown population standard deviation, $\frac{\bar{X}-\mu}{\frac{S}{\sqrt{n}}}$ is exactly t_{n-1}. A t calculation example (illustrative measurements) is:

```
x <- c(9, 10, 12, 11, 8)
mu0 <- 9
n <- length(x)
t_stat <- (mean(x) - mu0) / (sd(x) / sqrt(n))
pt(t_stat, df = n - 1, lower.tail = FALSE) # upper-tail probability
# Helpful extension: two-sided 95% confidence interval for the mean
mean(x) + c(-1, 1) * qt(0.975, df = n - 1) * sd(x) / sqrt(n)
```

Practice Sheet 5 Q5: one Weibull circuitboard has shape 2, scale 5. Its sample mean for $n = 50$ is approximately normal by the CLT:

```
beta <- 2
eta <- 5
n <- 50
mu <- eta * gamma(1 + 1 / beta)
sigma <- eta * sqrt(gamma(1 + 2 / beta) - gamma(1 + 1 / beta)^2)
pweibull(5, shape = beta, scale = eta, lower.tail = FALSE) # one: 0.367879
pnorm(5, mean = mu, sd = sigma / sqrt(n), lower.tail = FALSE) # mean: about 0.0412
ceiling((qnorm(0.975) * sigma / 0.5)^2) # 83 boards, CLT approximation
```

The small difference from the printed answer 0.0409 comes from rounded Gamma values/Z-table entries. Keep full precision in R. The "$n \geq 30$" rule is a course heuristic; strongly skewed/heavy-tailed populations can need larger samples. A strict requirement $n > a$ uses `floor(a) + 1`; `ceiling(a)` is sufficient for $n \geq a$ and for the noninteger thresholds above.

9. CLT simulation: each slide function in context

`numeric()` allocates a zero-filled vector; `for` repeats code and is a language construct, not a statistical function:

```
numeric(3)                  # 0, 0, 0
out <- numeric(3)
for (i in 1:3) { out[i] <- i^2 }
out                         # 1, 4, 9
```

Exponential population with rate 2: population mean and SD are both 0.5. Use the **current sample** in each repetition. The slide's `sd(expn)` inside a loop generating a different object can accidentally use an old sample.

```
set.seed(5507)
n <- 40
m <- 2000
```

```r
lambda <- 2
mu <- 1 / lambda
sigma <- 1 / lambda
sample_means <- numeric(m)
z_means <- numeric(m)
for (i in 1:m) {
  draw <- rexp(n, rate = lambda)
  sample_means[i] <- mean(draw)
  z_means[i] <- (mean(draw) - mu) / (sigma / sqrt(n))
}
hist(sample_means, freq = FALSE, nclass = 30)
lines(density(sample_means), col = "red")
hist(z_means, freq = FALSE, nclass = 30)
curve(dnorm(x), add = TRUE, col = "blue")
c(mean = mean(z_means), sd = sd(z_means)) # approximately 0 and 1
```

For a normal population and unknown sigma, use `sd(draw)` in the denominator to simulate a t statistic. Using a fixed known sigma simulates Z even if a slide comment says "sigma unknown".

Lognormal CLT example from Week 5, with corrected parameter names. **Always pass log-scale parameters to `rlnorm`**, including in the unknown-sigma simulation:

```r
set.seed(5507)
n <- 100
m <- 2000
meanlog <- 2
sdlog <- 0.2
mu <- exp(meanlog + sdlog^2 / 2)
sigma <- sqrt((exp(sdlog^2) - 1) * exp(2 * meanlog + sdlog^2))
z_means <- numeric(m)
studentised_means <- numeric(m)
for (i in 1:m) {
  draw <- rlnorm(n, meanlog = meanlog, sdlog = sdlog)
  z_means[i] <- (mean(draw) - mu) / (sigma / sqrt(n))
  studentised_means[i] <- (mean(draw) - mu) / (sd(draw) / sqrt(n))
}
c(mean = mean(z_means), sd = sd(z_means))
c(mean = mean(studentised_means), sd = sd(studentised_means))
```

The studentised lognormal mean is approximately normal for large n; it is not exactly t, because the parent distribution is not normal. For Weibull simulations replace the generator with `rweibull(n, shape = 2, scale = 3)` and calculate its population mean/SD with `gamma()` as in section 5.

10. Coverage index and checks

Course source	Functions/constructs covered here
Lecture 1 and Practice Sheet 1	`c`, `mean`, `median`, `var`, `sd`, `summary`, `head`, `options`, `stem`, `boxplot`, `hist`, `curve`, `text`, `set.seed`, `dnorm`, `pnorm`, `qnorm`, `rnorm`, `pbinom`, `qbinom`, `rbinom`.
Practice Sheets 2–3	`function`, `plot`, `expression`, `exp`, `log`, `gamma`, `dweibull`; probability, density, reliability and MTTF calculations.
Model Estimation slides and Practice Sheet 4	`getwd`, `read.csv`, `read.table`, `read_xlsx`, `write.csv`, `write.table`, `head`, `class`, `names`, `subset`, `sum`, `hist`, `dexp`, `install.packages`, `library`, `Surv`, `survfit`, `summary`, `plot`, `weibull.mle`, `estimate_cdf`, `plot_prob`.
CLT slides and Practice Sheet 5	`numeric`, `for`, `sample`, `length`, `mean`, `sd`, `sqrt`, `exp`, `gamma`, `hist`, `lines`, `density`, `rexp`, `rweibull`, `rlnorm`, `rnorm`, `pnorm`, `qnorm`, `pt`, `qt`.
Helpful extensions (not all explicitly called in slides)	Remaining `d/p/q/r` family members; `abs`, `choose`, `factorial`, `ceiling`, `floor`, `round`, `seq`, `rep`, `prod`, `sort`, `min`, `max`, `range`, `is.na`, `ifelse`, `data.frame`, `tempfile`, `read_excel`, `reliability_data`, `survreg`, `coef`.

Mathematical notation such as $F(t)$, $\mathrm{Var}(X)$ and the word "function" in prose is not automatically an R command. The inventory above filters these false positives.

Final calculation checks: probabilities must be between zero and one; `sd` takes a standard deviation; an exponential `rate` is the reciprocal of mean life; a Weibull `scale` is not generally MTTF; suspended observations retain their time exposure; sample-mean probabilities use the standard error. Keep units consistent and avoid overwriting useful names such as `mean`, `sd` or `c`.

Verification: all uncommented R code blocks were extracted and run successfully with Rscript, including plots, file round-trips, simulations and `survival` fits. Optional package and missing-file examples are explicitly commented and were checked against documentation rather than executed.